

Módulo: Programación Multimedia y dispositivos móviles. Unidad de Trabajo N°1: Introducción al desarrollo de aplicaciones móviles.

Actividad de Clase 7: Fragments.

1. Objetivo general.

Conocer, manejar y generar aplicaciones que tengan Fragments.

2. Metodología.

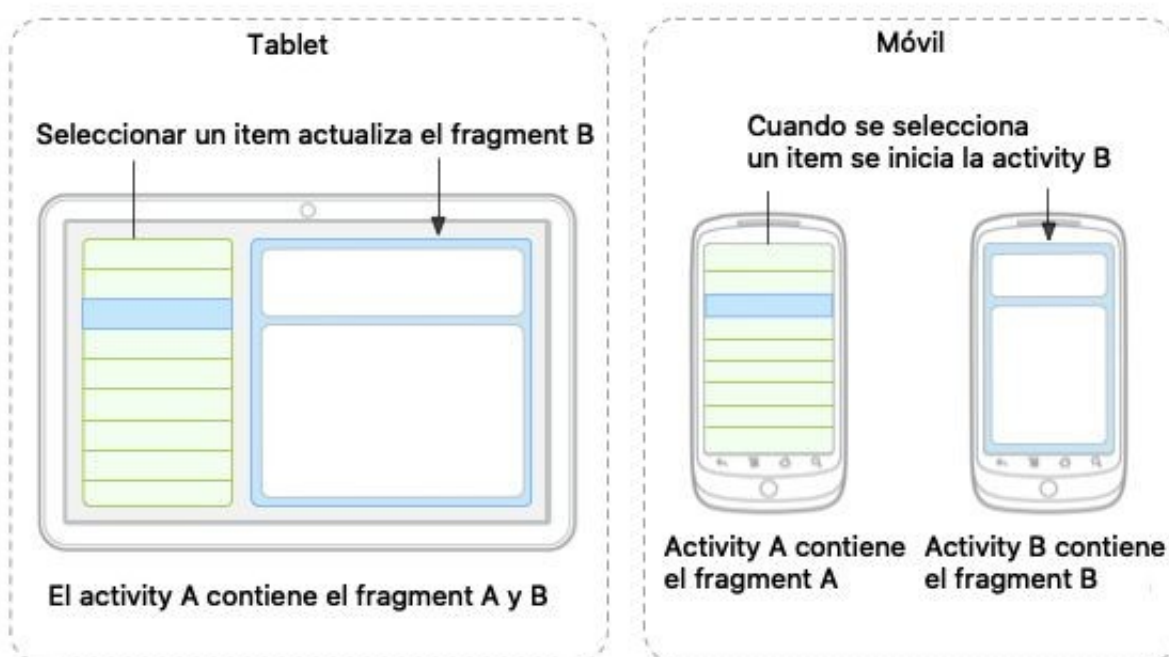
Cada alumno realizará la app.

3. Descripción.

Introducción: ¿Qué son los Fragments?.

Los **fragments**, representan un componente básico a la hora de desarrollar una app. A día de hoy conocemos las **activities**. Ahora imaginemos que el fragment es una variante de la activity pero con menos potencia, pero más manejable, es decir, su comportamiento es muy similar (tienen un layout y una clase para programar) pero tienen que estar dentro de una activity. La diferencia reside en que este componente es más reutilizable, y se puede añadir junto a más fragments en una misma activity.

Su popularidad empezó con la moda de las tablets, eran pantallas muy grandes que permitían aprovechar más los fragments añadiendo varios en una sola activity, mostrando más información que en un móvil.



A continuación puedes observar una imagen donde podemos ver las diferencias entre el ciclo de vida de un Activity y de un Fragment

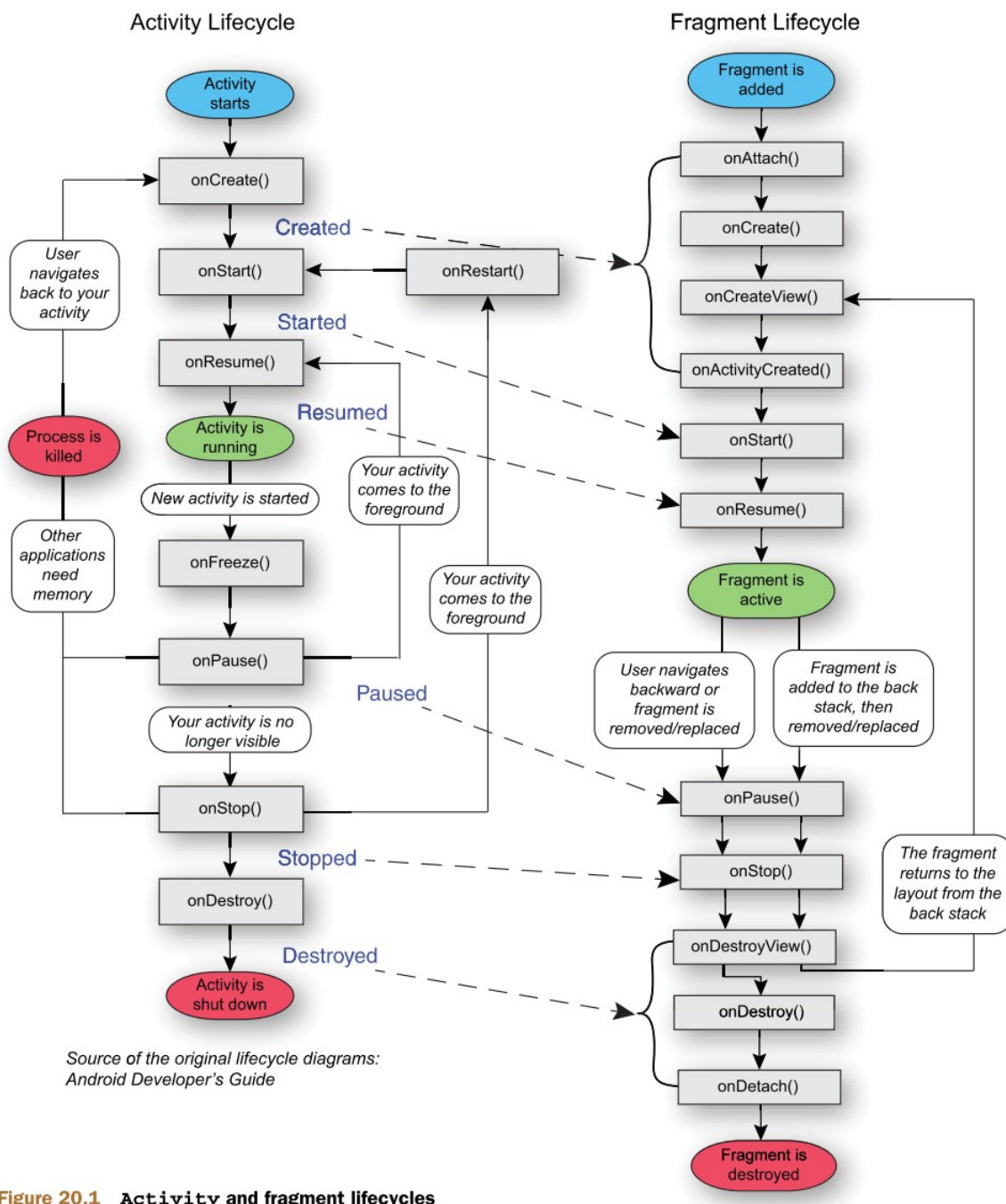


Figure 20.1 Activity and fragment lifecycles

Donde observamos la diferencia en **onCreate()** que en el Fragment se divide en

- **onAttach()** → se llama una vez que el fragmento está asociado con su actividad,
- **onCreateView** → crea y devuelve el Fragmento asociado.

- **onActivityCreated** --> Cuando se ha creado la actividad del fragmento y se crea una instancia de la jerarquía de vistas del fragmento.

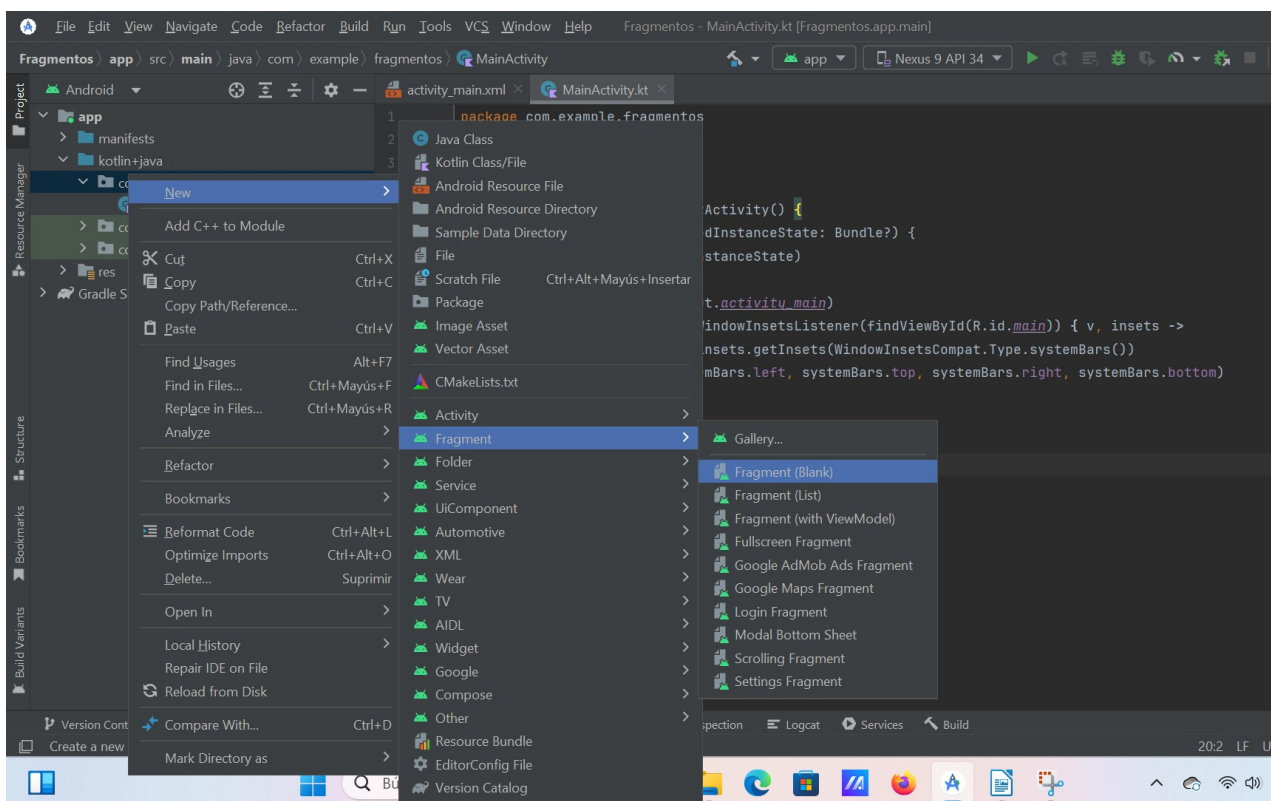
Y pasa lo mismo en onDestroy()

- **onDestroyView ()**: Fragmento limpia los recursos relacionados con su Vista.
- **onDestroy ()** se llama cuando el fragmento ya no se usa. Finalmente, se limpian todos los estados del fragmento.
- **onDetach ()**: el fragmento no está asociado con su actividad (digamos que lo desliga)

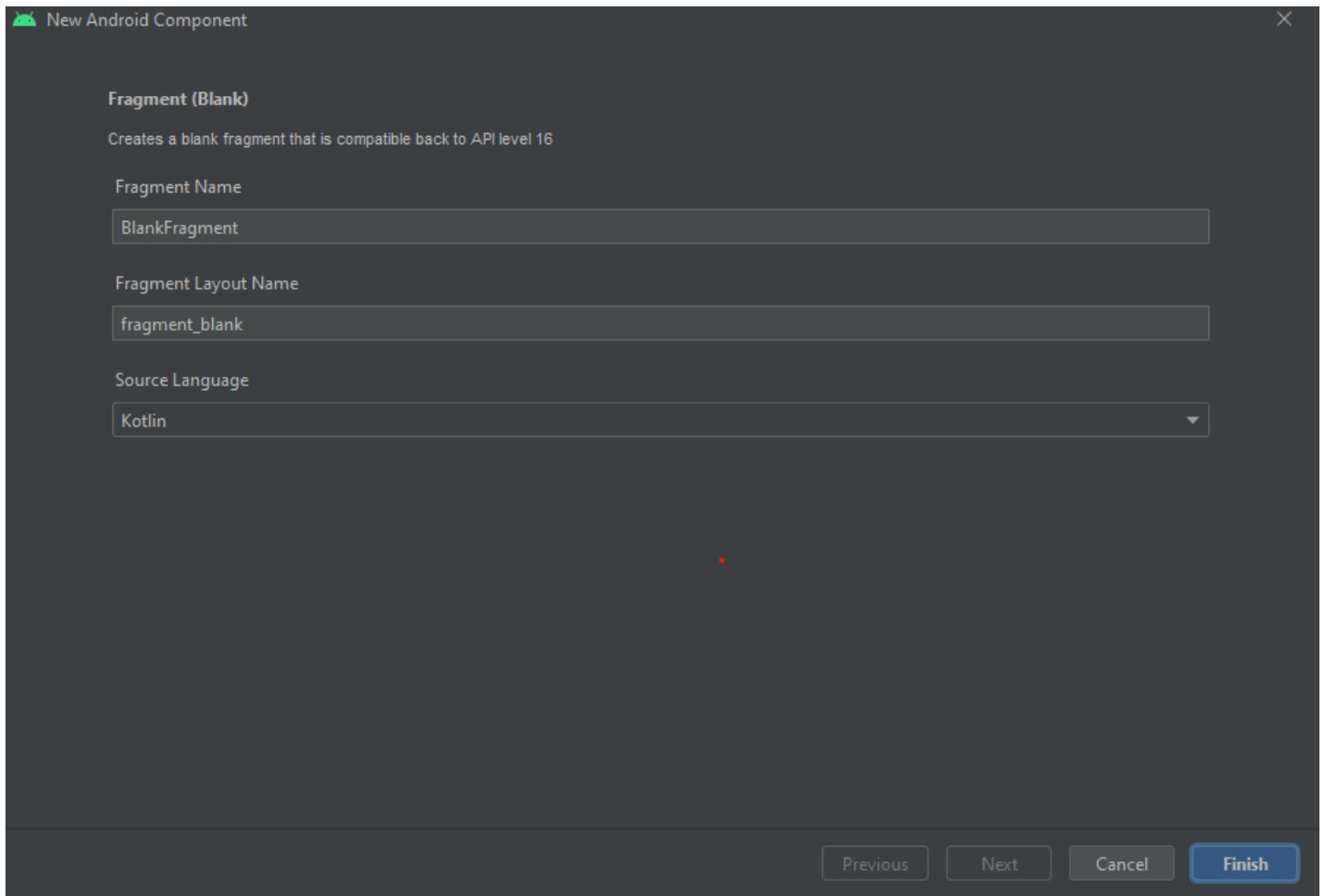
Creación de un aplicación que use Fragments.

Realiza lo siguiente:

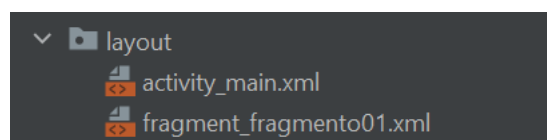
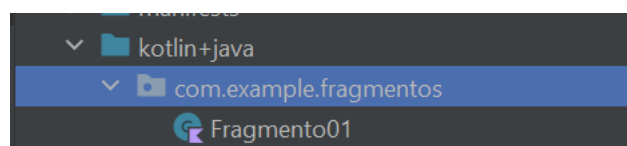
1) Realiza un aplicación que utilice Fragments para ello puede seguir los siguientes pasos. Crea un nuevo Proyecto. Una vez creado vamos crear un nuevo fragment Para ello pulsamos el botón derecho luego New → Fragment → Fragment (Blank)



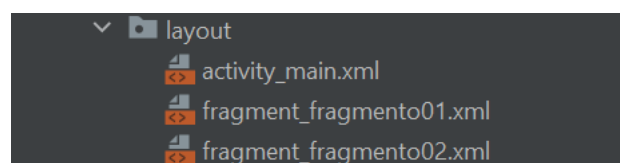
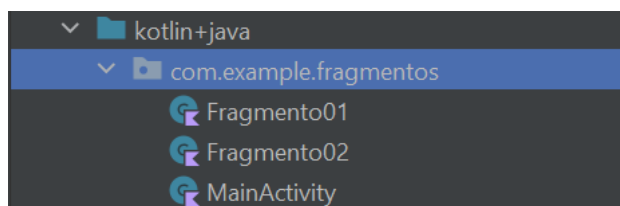
2) Nos aparecerá la siguiente ventana donde pondremos el nombre de Fragment y Fragment Layout damos a **Finish**



3) Esto nos creará un clase Kotlin y un fichero xml. A continuación puedes ver los que he creado.



4) Repite los pasos creando otro Fragment. Y al finalizar deberás tener otra clase más y otro fichero xml más. En la siguiente imagen puedes ver un ejemplo.



5) Ahora vamos a modificar cada uno de los fragments. Editaremos el primero y en la siguiente imagen puedes ver un ejemplo del aspecto del primer fragment. En este caso hemos utilizado un **LinearLayout**, que es un grupo de vistas que alinea todos los elementos secundarios en una única dirección, de manera vertical u horizontal. Puedes especificar la dirección del diseño con el atributo `android:orientation`. En nuestro caso hemos elegido vertical. Lo único que hemos hecho es poner la orientación vertical y cambiado el color de fondo de **fragment**. Y hemos añadido un **TextView** al que hemos puesto negrita y aumentado el tamaño de letra.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/material_dynamic_primary50"
    android:orientation="vertical"
    tools:context=".Fragmento01">

    <TextView
        android:id="@+id/tvfragmento1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Fragmento01"
        android:textSize="20sp"
        android:textStyle="bold" />

</LinearLayout>
```

Ahora vemos el contenido del segundo fragment. Que es similar al anterior utilizando `LinearLayout` y sólo un `TextView`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/material_dynamic_primary90"
    android:orientation="vertical"
    tools:context=".Fragmento02">

    <TextView
        android:id="@+id/tvfragmento2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Fragmento02"
        android:textSize="30sp"
        android:textStyle="bold" />

</LinearLayout>
```

6) Ahora vamos a modificar el layout de nuestra activity (**activity_main.xml**). En dicho layout vamos a utilizar **LinearLayout** (también con orientación vertical), luego crearemos otro **LinearLayout** (este con orientación horizontal) y dentro de este, dos botones (uno para mostrar el primer fragment y otro para mostrar el otro fragment. Y luego un **FrameLayout** donde mostraremos cada uno de los dos fragments según vayamos pulsando un botón u otro (este debe estar en el primer `LinearLayout`). En la siguiente imagen puedes ver un ejemplo del layout `activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">
```

```
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnfra1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Fragmento1" />
```

```
        <Button
            android:id="@+id/btnfra2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Fragmento2" />
```

```
    </LinearLayout>
```

```
    <FrameLayout
        android:id="@+id/fragmentoprin"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
```

```
    </FrameLayout>
```

```
</LinearLayout>
```

7) Una vez modificados todos los layouts vamos a modificar el código. Vamos a comenzar con la clase de kotlin asociada al primer fragment. Vamos a modificar sólo la función **onCreateView**. En ella vamos a utilizar

```
override fun onCreateView(  
    inflater: LayoutInflater, container: ViewGroup?,  
    savedInstanceState: Bundle?  
) : View? {  
    // Inflate the layout for this fragment  
    // Creamos un objeto donde inflamos y mostramos el contenido del fichero xml  
    val vista = inflater.inflate(R.layout.fragment_fragmento01, container, attachToRoot: false)  
  
    // Accedemos al TextView que contiene el Fragment 1  
    val tvfra1 = vista.findViewById<TextView>(R.id.tvfragmento1)  
  
    // Capturamos el evento click sobre el TextView  
    tvfra1.setOnClickListener {  
        Toast.makeText(activity, text: "Has hecho click en el TextView del Fragment 1", Toast.LENGTH_LONG).show()  
    }  
    return vista  
}
```

8) Se ha modificado el código que viene al crear el fragment. En el código que viene por defecto es **return** de **inflater.inflate(R.layout.fragment_fragmento01, container, false)**.

La clase LayoutInflater se utiliza para instanciar el archivo XML de presentación en sus objetos de vista correspondientes. En otras palabras, toma como entrada un archivo XML y genera los objetos Ver de él.

- 9) En nuestro ejemplo he almacenado el resultado en una vista para de esta forma poder acceder a los componentes que están definidos en el fragment en este ejemplo es sólo un **TextView** que luego accedo a él mediante **findViewById**. Sino lo hiciera así no podríamos acceder directamente a los componentes.
- 10) En el código lo único que hacemos además de acceder al TextView definir un listener de forma que cuando se haga click en el TextView aparezca un mensaje en el Toast. Como estamos en un fragment utilizamos activity (proviene de getActivity()) que es de la que depende el Fragment en vez de this.
- 11) Modifica de la misma forma la clase de Kotlin asociada al otro fragment. (adaptándose a los diferentes componentes y sus nombres)
- 12) Una vez modificada está clase. Vamos a modificar el código de nuestra **MainActivity**. En primer lugar accedemos a los dos botones que nos mostraran cada uno de los **fragment** mediante **findViewById** y luego definiremos un listener para cada botón

donde crearemos un objeto Fragment y a continuación llamamos a una función (método) que hará que se muestre el fragment correspondiente.

```
// Accedemos a los botones
val botonfra1 = findViewById<Button>(R.id.btnfra1)
val botonfra2 = findViewById<Button>(R.id.btnfra2)

// Definimos los listener a los botones

botonfra1.setOnClickListener {
    val fra1 = Fragmento01()
    showfragment(fra1)
}

botonfra2.setOnClickListener {
    val fra2 = Fragmento02()
    showfragment(fra2)
}
```

13) Ahora puedes ver el contenido de la función que muestra cada fragment, dicha función tiene que definirse fuera del **onCreate** pero dentro de la clase **MainActivity**.

```
fun showfragment(fragmento: Fragment) {
    val frag = supportFragmentManager.beginTransaction()
    frag.replace(R.id.fragmentoprin, fragmento)
    frag.commit()
}
```

14) Esta función recibe un **fragment** que será el que queramos cargar en nuestro **FrameLayout** del xml. Primero creamos un objeto **fragmentTransaction**, este objeto se crea con **supportFragmentManager** que es el que se encarga de gestionar los fragments y posee la mayoría de los métodos que necesitamos en la gestión.

15) Para cargar un fragment haremos siempre lo mismo, necesitamos avisar al sistema de que vamos a hacer un cambio, añadirlo y guardar dicho cambio. Eso es exactamente lo que hace nuestra función anterior. Empezamos avisando del cambio con **beginTransaction()**, luego llamamos a la función **replace()** del fragmentTransaction y lo cerramos con **commit()**.

Nota: yo no lo he utilizado pero a veces se usa para terminar antes de hacer el **commit()**, añadir **addToBackStack(null)**, que con esto le decimos al sistema que permita ir para atrás.

16) A continuación puedes observar una imagen de todo el código de MainActivity. Como puedes observar la función muestraFragmento está definido fuera de la función **onCreate** pero dentro de la Clase **MainActivity**.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContentView(R.layout.activity_main)  
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->  
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())  
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)  
            insets  
        }  
        // Accedemos a los botones  
        val botonfra1 = findViewById<Button>(R.id.btnfra1)  
        val botonfra2 = findViewById<Button>(R.id.btnfra2)  
  
        // Definimos los listener a los botones  
        botonfra1.setOnClickListener {  
            val fra1 = Fragmento01()  
            showfragment(fra1)  
        }  
        botonfra2.setOnClickListener {  
            val fra2 = Fragmento02()  
            showfragment(fra2)  
        }  
    }  
    fun showfragment(fragmento: Fragment) {  
        val frag = supportFragmentManager.beginTransaction()  
        frag.replace(R.id.fragmentoprin, fragmento)  
        frag.commit()  
    }  
}
```

Paso de parámetros desde la Activity al Fragments.

Modificaremos la app añadiendo un spinner. El cuál cuando cambiemos el ítem seleccionado este aparezca en una de los fragments (el que nosotros seleccionemos. Para ello realiza lo siguiente:

17) Vamos a añadir un spinner a nuestra **MainActivity** (justo encima del **FrameLayout**).

```
<Spinner
    android:id="@+id/espiner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />

<FrameLayout
    android:id="@+id/fragmentoprin"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

</FrameLayout>
```

18) En el **MainActivity**, vamos a realizar lo siguiente. Crea el array con los datos que van a aparecer en el Spinner y el Adapter para vincular los datos. Y definimos el listener sobre el spinner donde almacenarás el ítem seleccionado, puedes recordar como hacerlo consultado la actividad de spinner.

19) Una vez realizada esta parte, nos vamos al código del **Fragment** donde queramos aparezca el valor seleccionado. Y le damos nombre al/los parámetros que vamos a recibir. En este ejemplo se va a modificar sólo uno. Primero le damos nombre al parámetro, en primer lugar, lo añadimos después de los import antes de definir la clase y luego en la definición **private var**.....y luego en la función **onCreate** donde inicializamos dicho parámetro.

```
import ...

// TODO: Rename parameter arguments, choose names that match
// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
private const val ARG_PARAM1 = "lenguaje"
private const val ARG_PARAM2 = "param2"

/**
 * A simple [Fragment] subclass.
 * Use the [Fragmento01.newInstance] factory method to
 * create an instance of this fragment.
 */
class Fragmento01 : Fragment() {
    // TODO: Rename and change types of parameters
    private var lenguaje: String? = null
    private var param2: String? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        arguments?.let { it: Bundle
            lenguaje = it.getString(ARG_PARAM1)
            param2 = it.getString(ARG_PARAM2)
        }
    }
}
```

20) A continuación, sobrescribimos la función (o método) **onViewCreated** donde vamos a acceder al **TextView** definido en Fragment (a través del view), y le vamos a asignar como texto el valor de parámetro pasado desde la Activity.

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
    super.onViewCreated(view, savedInstanceState)  
    view.findViewById<TextView>(R.id.tvfragmento1).text = lenguaje  
}
```

21) Ahora vamos al **MainActivity** para implementar el paso de parámetros al **Fragment**. En nuestro caso vamos a seguir utilizando los botones para cambiar de **Fragment**. Por ello en el primer botón vamos a hacer algo parecido a cuando intercambiábamos datos con otra Activity. Vamos a crear un **Bundle**. Al cuál añadiremos el parámetro que hemos puesto en el Fragment y el valor seleccionado del spinner y se lo vamos a pasar como argumento al Fragment en el que va a aparecer el valor seleccionado en el spinner. Podemos ver un ejemplo a continuación.

```
// Definimos los listeners a los botones  
botonfra1.setOnClickListener {  
    val fra1 = Fragmento01()  
    val datos = Bundle()  
    datos.putString("lenguaje", seleccionado)  
    fra1.arguments=datos  
    showfragment(fra1)  
}
```

22) Una vez realizado esto comprueba el funcionamiento de tu aplicación.

23) Modifica la app para que el usuario no tenga que darle a botón de MOSTRAR FRAG1 y cada vez que cambie el valor en el spinner se actualice en Fragment correspondiente.